

Full Stack Development with MERN

Project Documentation

1. Introduction

Project Title

Stock Trading App

Team Members

Name	Role
AKKUPALLI SASI	Full Stack Developer(MERN)

2. Project Overview

Purpose

The Stock Trading App is a web-based application developed using the MERN stack. It allows users to securely register and log in, buy and sell stocks, maintain their investment portfolio, and view transaction history. The project demonstrates full-stack web development concepts including authentication, REST APIs, database management, and responsive user interface design.

Features

- User Registration
 - User Login using JWT Authentication
 - Secure Password Encryption
 - Buy Stocks
 - Sell Stocks
 - Portfolio Management
 - Transaction History
 - Dashboard Summary Cards
 - Profit/Loss Calculation
 - Responsive User Interface
 - Logout Functionality
-

3. Architecture

Frontend

The frontend is developed using **React.js**. It provides a responsive and interactive user interface where users can:

- Register/Login
- Buy Stocks
- Sell Stocks
- View Portfolio
- View Transaction History
- View Profit/Loss Summary

Technologies Used

- React.js
 - HTML5
 - CSS3
 - Axios
 - React Router DOM
-

Backend

The backend is built using **Node.js** and **Express.js**.

Responsibilities include:

- User Authentication
- JWT Token Generation
- Portfolio Management
- Stock Transactions
- Business Logic
- REST API Development
- MongoDB Operations

Technologies Used

- Node.js
 - Express.js
 - JWT
 - bcrypt
 - Mongoose
-

Database

MongoDB Atlas is used as the cloud database.

Collections

- Users
- Portfolios
- Transactions

MongoDB stores:

- User Details
 - Portfolio Information
 - Stock Holdings
 - Transaction History
-

4. Setup Instructions

Prerequisites

Install the following software:

- Node.js (v18 or above)
 - npm
 - MongoDB Atlas Account
 - Git
 - VS Code
-

Installation

Step 1

Clone the repository

```
git clone https://github.com/yourusername/stock-trading-app.git
```

Step 2

Navigate into project folder

```
cd stock-trading-app
```

Step 3

Install backend dependencies

```
cd server  
npm install
```

Step 4

Install frontend dependencies

```
cd ../client
npm install
```

Step 5

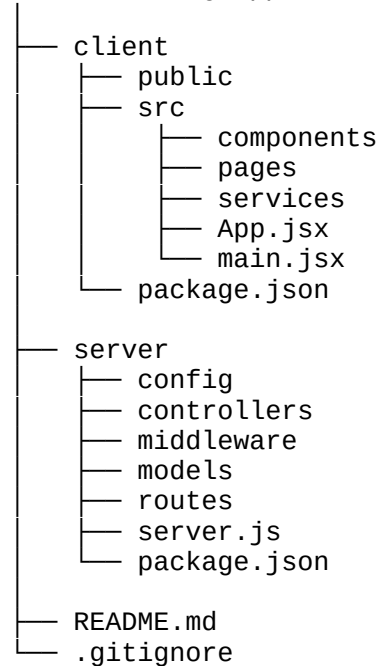
Create a **.env** file inside the server folder.

Example:

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_secret_key
FINNHUB_API_KEY=your_api_key
```

5. Folder Structure

Stock-Trading-App



6. Running the Application

Backend

```
cd server
```

```
npm run dev
```

Backend runs at

<http://localhost:5000>

Frontend

```
cd client  
npm run dev
```

Frontend runs at

<http://localhost:5173>

7. API Documentation

Authentication APIs

Register

POST /api/auth/register

Request

```
{  
  "name": "John",  
  "email": "john@gmail.com",  
  "password": "123456"  
}
```

Response

```
{  
  "message": "User Registered Successfully"  
}
```

Login

POST /api/auth/login

Request

```
{  
  "email": "john@gmail.com",  
  "password": "123456"  
}
```

Response

```
{
  "token": "JWT_TOKEN"
}
```

Buy Stock

POST /api/trade/buy

Request

```
{
  "symbol": "AAPL",
  "quantity": 5,
  "price": 200
}
```

Response

```
{
  "message": "Stock Purchased Successfully"
}
```

Sell Stock

POST /api/trade/sell

Request

```
{
  "symbol": "AAPL",
  "quantity": 2,
  "price": 210
}
```

Response

```
{
  "message": "Stock Sold Successfully"
}
```

Portfolio

GET /api/portfolio

Returns all portfolio details.

Transactions

GET /api/transactions

Returns transaction history.

8. Authentication

Authentication is implemented using **JWT (JSON Web Token)**.

Workflow:

- User registers
- Password is encrypted using bcrypt
- User logs in
- JWT token is generated
- Token is stored in Local Storage
- Protected APIs verify the token before granting access

Security Features

- Password Hashing
 - JWT Authentication
 - Protected Routes
 - Unauthorized Access Prevention
-

9. User Interface

The application contains the following pages:

- Login Page
- Registration Page
- Dashboard
- Buy Stock Form
- Sell Stock Form
- Portfolio Table
- Transaction History
- Dashboard Summary Cards
- Logout Button

(Insert screenshots of each page here.)

10. Testing

The application was tested manually for all major functionalities.

Test Cases

- User Registration

- User Login
- Buy Stock
- Sell Stock
- Portfolio Update
- Transaction History
- Logout
- Invalid Login
- Invalid Stock Quantity
- JWT Authentication

Tools Used

- Postman
 - Browser Developer Tools
 - MongoDB Atlas
-

11. Screenshots / Demo

Include screenshots of:

- Registration Page
- Login Page
- Dashboard
- Buy Stock
- Sell Stock
- Portfolio
- Transaction History

GitHub Repository

<https://github.com/yourusername/stock-trading-app>

Demo Video (Optional)

<https://youtube.com/...>

12. Known Issues

- Real-time stock prices depend on the external Stock API.
 - Internet connection is required to access MongoDB Atlas.
 - API rate limits may restrict frequent stock requests.
 - Admin module is not implemented.
-

13. Future Enhancements

- Real-Time Stock Price Updates
 - Watchlist Feature
 - Stock Charts
 - Portfolio Analytics
 - Email Notifications
 - Password Reset
 - User Profile Management
 - Admin Dashboard
 - Cloud Deployment (Render/Vercel)
 - Dark Mode
 - Mobile Application
 - Multi-Currency Support
 - AI-Based Stock Recommendation
 - Export Portfolio as PDF/Excel
-

Technologies Used

Category	Technology
Frontend	React.js, HTML5, CSS3
Backend	Node.js, Express.js
Database	MongoDB Atlas
Authentication	JWT, bcrypt
API Testing	Postman
Version Control	Git, GitHub
IDE	Visual Studio Code
Runtime	Node.js
HTTP Client	Axios

Conclusion

The **Stock Trading App** successfully demonstrates the implementation of a complete MERN stack application with secure authentication, portfolio management, stock trading functionality, RESTful APIs, and MongoDB integration. The project provides a responsive and user-friendly interface while following modern full-stack development practices, making it suitable as an academic project and a strong portfolio project.